

Module	Description	Example	Script
anthropic	creating a client	client = Anthropic(api_key=apikey)	g20/demo.py
anthropic	importing the module	from anthropic import Anthropic	g20/demo.py
core	api, reading a key from a file	apikey = fh.readline().strip()	g19/demo.py
core	class, accessing the instance	cols = self.columns	g20/demo.py
core	class, defining a class attribute	sig = 'Howdy'	g20/demo.py
core	class, defining a method	def capcols(self):	g20/demo.py
core	class, extending existing class	class myDF(pd.DataFrame):	g20/demo.py
core	continue, going on to next loop item	continue	g06/demo.py
core	dictionary, adding a new entry	co['po'] = 'CO'	g05/demo.py
core	dictionary, creating	co = {'name':'Colorado', 'capital':'Denver'}	g05/demo.py
core	dictionary, creating via comprehension	word_lengths = { w:len(w) for w in wordlist }	g06/demo.py
core	dictionary, iterating through key-value pairs	for w,l in word_lengths.items():	g06/demo.py
core	dictionary, looking up a value	name = ny['name']	g05/demo.py
core	dictionary, making a list of	list1 = [co,ny]	g05/demo.py
core	dictionary, obtaining a list of keys	names = super_dict.keys()	g05/demo.py
core	f-string, grouping with commas	print(f'Total population: {tot_pop:,}')	g12/demo.py
core	f-string, using a formatting string	print(f"PV of {payment} with T={year} and r={r} is \${p...	g08/demo.py
core	file, closing	fh.close()	g02/demo.py
core	file, opening for reading	fh = open('states.csv')	g05/demo.py
core	file, opening for writing	fh = open(filename,"w")	g02/demo.py
core	file, output using print	print("It was written during",year,file=fh)	g02/demo.py
core	file, output using write	fh.write("Where was this file was written?\n")	g02/demo.py
core	file, print without adding spaces	print('\nOuter:\n', join_o['_merge'].value_counts(), s...	g15/demo.py
core	file, reading one line at a time	for line in fh:	g05/demo.py
core	for, looping through a list	for n in a_list:	g04/demo.py
core	for, looping through a list of tuples	for number,name in div_info:	g14/demo.py
core	function, calling	d1_ssqr = sumsq(d1)	g07/demo.py
core	function, calling with an optional argument	sample_function(100, 10, r=0.07)	g08/demo.py
core	function, defining	def sumsq(values: list) -> float:	g07/demo.py

Module	Description	Example	Script
core	function, defining with optional argument	<code>def sample_function(payment:float,year:int,r:float=0.05...</code>	g08/demo.py
core	function, returning a result	<code>return values</code>	g07/demo.py
core	function, using type hinting	<code>def readlist(filename: str) -> list:</code>	g07/demo.py
core	if, starting a conditional block	<code>if l == 5:</code>	g06/demo.py
core	if, using an elif statement	<code>elif s.isalpha():</code>	g06/demo.py
core	if, using an else statement	<code>else:</code>	g06/demo.py
core	list, appending an element	<code>a_list.append("four")</code>	g03/demo.py
core	list, create via comprehension	<code>cubes = [n**3 for n in a_list]</code>	g04/demo.py
core	list, creating	<code>a_list = ["zero", "one", "two", "three"]</code>	g03/demo.py
core	list, determining length	<code>n = len(b_list)</code>	g03/demo.py
core	list, extending with another list	<code>a_list.extend(a_more)</code>	g03/demo.py
core	list, generating a sequence	<code>b_list = range(1,6)</code>	g04/demo.py
core	list, joining with spaces	<code>a_string = " ".join(a_list)</code>	g03/demo.py
core	list, selecting an element	<code>print(a_list[0])</code>	g03/demo.py
core	list, selecting elements 0 to 3	<code>print(a_list[:4])</code>	g03/demo.py
core	list, selecting elements 1 to 2	<code>print(a_list[1:3])</code>	g03/demo.py
core	list, selecting elements 1 to the end	<code>print(a_list[1:])</code>	g03/demo.py
core	list, selecting last 3 elements	<code>print(a_list[-3:])</code>	g03/demo.py
core	list, selecting the last element	<code>print(a_list[-1])</code>	g03/demo.py
core	list, sorting	<code>c_sort = sorted(b_list)</code>	g03/demo.py
core	list, summing	<code>total = sum(numbers)</code>	g06/demo.py
core	math, raising a number to a power	<code>a_cubes.append(n**3)</code>	g04/demo.py
core	math, rounding a number	<code>rounded = round(ratio,2)</code>	g05/demo.py
core	sets, computing difference	<code>print(name_states - pop_states)</code>	g14/demo.py
core	sets, creating	<code>name_states = set(name_data['State'])</code>	g14/demo.py
core	sets, of tuples	<code>tset1 = set([(1,2), (2,3), (1,3), (2,3)])</code>	g14/demo.py
core	string, concatenating	<code>name = s1+" "+s2+" "+s3</code>	g02/demo.py
core	string, convert to lower case	<code>lower = [s.lower() for s in wordlist]</code>	g06/demo.py
core	string, convert to title case	<code>new_s = s.title()</code>	g06/demo.py
core	string, converting to an int	<code>value = int(s)</code>	g06/demo.py
core	string, creating	<code>filename = "demo.txt"</code>	g02/demo.py
core	string, finding starting index	<code>mm_start = long_string.find("mm")</code>	g06/demo.py

Module	Description	Example	Script
core	string, including a newline character	<code>fh.write(name+"!\n")</code>	g02/demo.py
core	string, is entirely numeric	<code>if s.isnumeric():</code>	g06/demo.py
core	string, matching a substring	<code>has_ñ = [s for s in lower if "ñ" in s]</code>	g06/demo.py
core	string, matching end	<code>a_end = [s for s in lower if s.endswith("a")]</code>	g06/demo.py
core	string, matching multiple starts	<code>ab_start = [s for s in lower if s.startswith(starters)]</code>	g06/demo.py
core	string, matching start	<code>a_start = [s for s in lower if s.startswith("a")]</code>	g06/demo.py
core	string, replacing a substring	<code>words = s.replace(","," ").split()</code>	g06/demo.py
core	string, splitting on a comma	<code>parts = line.split(',')</code>	g05/demo.py
core	string, splitting on whitespace	<code>b_list = b_string.split()</code>	g03/demo.py
core	string, stripping blank space	<code>clean = [item.strip() for item in parts]</code>	g05/demo.py
core	string, using the format method	<code>my_ask = prompt.format(tidy)</code>	g20/demo.py
core	tuple, creating	<code>starters = ("a","b","0")</code>	g06/demo.py
core	type, obtaining for a variable	<code>print('\nraw_states is a DataFrame object:', type(raw_...</code>	g10/demo.py
csv	setting up a DictReader object	<code>reader = csv.DictReader(fh)</code>	g09/demo.py
json	importing the module	<code>import json</code>	g05/demo.py
json	using to print an object nicely	<code>print(json.dumps(list1,indent=4))</code>	g05/demo.py
matplotlib	axes, adding a horizontal line	<code>ax21.axhline(medians['etr'], c='r', ls='-', lw=1)</code>	g13/demo.py
matplotlib	axes, adding a vertical line	<code>ax21.axvline(medians['inc'], c='r', ls='-', lw=1)</code>	g13/demo.py
matplotlib	axes, labeling the X axis	<code>ax.set_xlabel('Millions')</code>	g12/demo.py
matplotlib	axes, labeling the Y axis	<code>ax.set_ylabel('Millions')</code>	g12/demo.py
matplotlib	axes, turning off a label	<code>ax.set_ylabel(None)</code>	g14/demo.py
matplotlib	figure, adding a title	<code>fig2.suptitle('Pooled Data')</code>	g13/demo.py
matplotlib	figure, four panel grid	<code>fig3, axs = plt.subplots(2,2,sharex=True,sharey=True)</code>	g13/demo.py
matplotlib	figure, left and right panels	<code>fig2, (ax21,ax22) = plt.subplots(1,2)</code>	g13/demo.py
matplotlib	figure, saving	<code>fig.savefig('figure.png')</code>	g12/demo.py
matplotlib	figure, setting the size	<code>fig, axs = plt.subplots(1,2,figsize=(12,6))</code>	g21/demo.py
matplotlib	figure, tuning the layout	<code>fig.tight_layout()</code>	g12/demo.py
matplotlib	figure, working with a list of axes	<code>for ax in axs:</code>	g21/demo.py
matplotlib	importing pyplot	<code>import matplotlib.pyplot as plt</code>	g12/demo.py
matplotlib	setting the default resolution	<code>plt.rcParams['figure.dpi'] = 300</code>	g12/demo.py
matplotlib	using subplots to set up a figure	<code>fig, ax = plt.subplots()</code>	g12/demo.py

Module	Description	Example	Script
os	importing the module	import os	g19/demo.py
os	test if a file or directory exists	if os.path.exists('apikey.txt'):	g19/demo.py
pandas	categorical variable, categories	print(cat.categories)	g13/demo.py
pandas	categorical variable, codes	tidy_data['cat_codes'] = cat.codes	g13/demo.py
pandas	categorical variable, creating	cat = pd.Categorical(tidy_data['type'])	g13/demo.py
pandas	columns, dividing along index	by_day_pct = 100*by_day_use.div(by_day_tot,axis='index'...	g18/demo.py
pandas	columns, dividing with explicit alignment	normed2 = 100*states.div(pa_row,axis='columns')	g10/demo.py
pandas	columns, listing names	print('\nColumns:', list(raw_states.columns))	g10/demo.py
pandas	columns, renaming	county = county.rename(columns={'B01001_001E':'pop'})	g11/demo.py
pandas	columns, retrieving one by name	pop = states['pop']	g10/demo.py
pandas	columns, retrieving several by name	print(pop[some_states]/1e6)	g10/demo.py
pandas	dataframe, appending	gen_all = pd.concat([gen_oswego, gen_onondaga])	g16/demo.py
pandas	dataframe, boolean row selection	print(trim[has_AM], "\n")	g13/demo.py
pandas	dataframe, dropping a column	both = both.drop(columns='_merge')	g16/demo.py
pandas	dataframe, dropping duplicates	flood = flood.drop_duplicates(subset='TAX_ID')	g15/demo.py
pandas	dataframe, dropping missing data	merged = geocodes.dropna()	g12/demo.py
pandas	dataframe, finding duplicate records	dups = parcels.duplicated(subset='TAX_ID', keep=False...	g15/demo.py
pandas	dataframe, getting a block of rows via index	sel = merged.loc[number]	g14/demo.py
pandas	dataframe, inner 1:1 merge	join_i = parcels.merge(flood, how='inner', on="TAX_ID",...	g15/demo.py
pandas	dataframe, inner join	merged = name_data.merge(pop_data,left_on="State",right...	g14/demo.py
pandas	dataframe, iterating through rows	for idx,rec in sample.iterrows():	g20/demo.py
pandas	dataframe, left 1:1 merge	join_l = parcels.merge(flood, how='left', on="TAX_ID",...	g15/demo.py
pandas	dataframe, left m:1 merge	both = gen_all.merge(plants, how='left', on='Plant Code...	g16/demo.py
pandas	dataframe, making a copy	trim = trim.copy()	g13/demo.py
pandas	dataframe, melting	long_form = means.reset_index().melt(id_vars='month')	g18/demo.py
pandas	dataframe, outer 1:1 merge	join_o = parcels.merge(flood, how='outer', on="TAX_ID",...	g15/demo.py
pandas	dataframe, pivoting	by_day_use = usage.pivot(index=['month','day'],columns=...	g18/demo.py
pandas	dataframe, reading tab-delimited data	raw = pd.read_csv(input_file,	g20/demo.py
pandas	dataframe, reading zipped pickle format	sample2 = pd.read_pickle('sample.pkl.zip')	g17/demo.py
pandas	dataframe, resetting the index	hourly = hourly.reset_index()	g18/demo.py
pandas	dataframe, right 1:1 merge	join_r = parcels.merge(flood, how='right', on="TAX_ID",...	g15/demo.py
pandas	dataframe, saving in zipped pickle format	sample.to_pickle('sample.pkl.zip')	g17/demo.py
pandas	dataframe, selecting rows by list indexing	print(low_to_high[-5:])	g10/demo.py
pandas	dataframe, selecting rows via boolean	dup_rec = flood[dups]	g15/demo.py

Module	Description	Example	Script
pandas	dataframe, selecting rows via query	trimmed = county.query("state == '04' or state == '36' ")	g11/demo.py
pandas	dataframe, selective drop of missing data	trim = demo.dropna(subset="Days")	g13/demo.py
pandas	dataframe, sorting by a column	county = county.sort_values('pop')	g11/demo.py
pandas	dataframe, sorting by index	summary = summary.sort_index(ascending=False)	g16/demo.py
pandas	dataframe, summing a boolean	print('\nduplicate parcels:', dups.sum())	g15/demo.py
pandas	dataframe, summing across columns	by_day_tot = by_day_use.sum(axis='columns')	g18/demo.py
pandas	dataframe, unstacking an index level	bymo = bymo.unstack('month')	g18/demo.py
pandas	dataframe, using a multilevel column index	means = grid['mean']	g21/demo.py
pandas	dataframe, using xs to select a subset	print(county.xs('04',level='state'))	g11/demo.py
pandas	dataframe, using xs with columns	c1 = grid.xs('c1',axis='columns',level=1)	g21/demo.py
pandas	dataframe, writing as an Excel file	trim.to_excel(output_file,index=False)	g20/demo.py
pandas	dataframe, writing to a CSV file	merged.to_csv('demo-merged.csv')	g14/demo.py
pandas	datetime, building via to_datetime()	date = pd.to_datetime(recs['ts'])	g15/demo.py
pandas	datetime, building with a format	ymd = pd.to_datetime(sample['TRANSACTION_DT'], format=...	g17/demo.py
pandas	datetime, extracting day attribute	recs['day'] = date.dt.day	g15/demo.py
pandas	datetime, extracting hour attribute	recs['hour'] = date.dt.hour	g15/demo.py
pandas	general, display information about object	sample.info()	g17/demo.py
pandas	general, displaying all columns	pd.set_option('display.max_columns',None)	g17/demo.py
pandas	general, displaying all rows	pd.set_option('display.max_rows', None)	g10/demo.py
pandas	general, importing the module	import pandas as pd	g10/demo.py
pandas	general, using copy_on_write mode	pd.options.mode.copy_on_write = True	g17/demo.py
pandas	general, using qcut to create deciles	dec = pd.qcut(county['pop'], 10, labels=range(1,11))	g11/demo.py
pandas	groupby, cumulative sum within group	cumulative_inc = group_by_state['pop'].cumsum()	g11/demo.py
pandas	groupby, descriptive statistics	inc_stats = group_by_state['pop'].describe()	g11/demo.py
pandas	groupby, iterating over groups	for t,g in group_by_state:	g11/demo.py
pandas	groupby, median of each group	pop_med = group_by_state['pop'].median()	g11/demo.py
pandas	groupby, quantile of each group	pop_25th = group_by_state['pop'].quantile(0.25)	g11/demo.py
pandas	groupby, return group number	groups = group_by_state.ngroup()	g11/demo.py
pandas	groupby, return number within group	seqnum = group_by_state.cumcount()	g11/demo.py
pandas	groupby, return rank within group	rank_pop = group_by_state['pop'].rank()	g11/demo.py
pandas	groupby, select first records	first2 = group_by_state.head(2)	g11/demo.py
pandas	groupby, select largest values	largest = group_by_state['pop'].nlargest(2)	g11/demo.py
pandas	groupby, select last records	last2 = group_by_state.tail(2)	g11/demo.py
pandas	groupby, size of each group	num_rows = group_by_state.size()	g11/demo.py

Module	Description	Example	Script
pandas	groupby, sum of each group	<code>state = county.groupby('state')['pop'].sum()</code>	g11/demo.py
pandas	index, creating with 3 levels	<code>county = county.set_index(['state', 'county', 'NAME'])</code>	g11/demo.py
pandas	index, listing names	<code>print('\nIndex (rows):', list(raw_states.index))</code>	g10/demo.py
pandas	index, renaming values	<code>div_pop = div_pop.rename(index=div_names)</code>	g12/demo.py
pandas	index, retrieving a row by name	<code>pa_row = states.loc['Pennsylvania']</code>	g10/demo.py
pandas	index, retrieving first rows by location	<code>print(low_to_high.iloc[0:10])</code>	g10/demo.py
pandas	index, retrieving last rows by location	<code>print(low_to_high.iloc[-5:])</code>	g10/demo.py
pandas	index, setting to a column	<code>states = raw_states.set_index('name')</code>	g10/demo.py
pandas	plotting, bar plot	<code>reg_pop.plot.bar(title='Population',ax=ax)</code>	g12/demo.py
pandas	plotting, histogram	<code>hh_data['etr'].plot.hist(ax=ax1, bins=20, title='Distribu. . .</code>	g13/demo.py
pandas	plotting, horizontal bar plot	<code>div_pop.plot.barh(title='Population',ax=ax)</code>	g12/demo.py
pandas	plotting, scatter colored by 3rd var	<code>tidy_data.plot.scatter(ax=ax4,x='Income',y='ETR',c='Reg. . .</code>	g13/demo.py
pandas	plotting, scatter plot	<code>hh_data.plot.scatter(ax=ax21,x='inc',y='etr',title='ETR. . .</code>	g13/demo.py
pandas	plotting, turning off legend	<code>sel.plot.barh(x='Name',y='percent',ax=ax,legend=None)</code>	g14/demo.py
pandas	reading, csv data	<code>raw_states = pd.read_csv('state-data.csv')</code>	g10/demo.py
pandas	reading, from an open file handle	<code>gen_oswego = pd.read_csv(fh1)</code>	g16/demo.py
pandas	reading, setting index column	<code>state_data = pd.read_csv('state-data.csv',index_col='na. . .</code>	g12/demo.py
pandas	reading, using dtype dictionary	<code>county = pd.read_csv('county_pop.csv',dtype=fips)</code>	g11/demo.py
pandas	series, RE at start	<code>is_LD = trim['Number'].str.contains(r"12")</code>	g13/demo.py
pandas	series, automatic alignment by index	<code>merged['percent'] = 100*merged['pop']/div_pop</code>	g14/demo.py
pandas	series, contains RE or RE	<code>is_TT = trim['Days'].str.contains(r"Tu Th")</code>	g13/demo.py
pandas	series, contains a plain string	<code>has_AM = trim['Time'].str.contains("AM")</code>	g13/demo.py
pandas	series, contains an RE	<code>has_AMPM = trim['Time'].str.contains("AM.*PM")</code>	g13/demo.py
pandas	series, converting strings to title case	<code>fixname = subset_view['NAME'].str.title()</code>	g17/demo.py
pandas	series, converting to a list	<code>print(name_data['State'].to_list())</code>	g14/demo.py
pandas	series, element-by-element or	<code>is_either = is_ca is_tx</code>	g17/demo.py
pandas	series, random sample	<code>vins = vins.sample(frac=0.001,random_state=789)</code>	g20/demo.py
pandas	series, retrieving an element	<code>print("\nFlorida's population:", pop['Florida']/1e6)</code>	g10/demo.py
pandas	series, sort in decending order	<code>div_pop = div_pop.sort_values(ascending=False)</code>	g12/demo.py
pandas	series, sorting by value	<code>low_to_high = normed['med_pers_inc'].sort_values()</code>	g10/demo.py
pandas	series, splitting via RE	<code>trim['Split'] = trim["Time"].str.split(r": - ")</code>	g13/demo.py
pandas	series, splitting with expand	<code>exp = trim["Time"].str.split(r": - ", expand=True)</code>	g13/demo.py
pandas	series, summing	<code>reg_pop = by_reg['pop'].sum()/1e6</code>	g12/demo.py

Module	Description	Example	Script
pandas	series, unstacking	<code>tot_wide = tot_amt.unstack('PGI')</code>	<code>g17/demo.py</code>
pandas	series, using <code>isin()</code>	<code>fixed = flood['TAX_ID'].isin(dup_rec['TAX_ID'])</code>	<code>g15/demo.py</code>
pandas	series, using <code>value_counts()</code>	<code>print('\nOuter:\n', join_o['_merge'].value_counts(), s...</code>	<code>g15/demo.py</code>
pydantic	defining an object	<code>class ComplaintInfo(BaseModel):</code>	<code>g20/demo.py</code>
pydantic	importing the module	<code>from pydantic import BaseModel</code>	<code>g20/demo.py</code>
requests	calling the <code>get()</code> method	<code>response = requests.get(api,payload)</code>	<code>g19/demo.py</code>
requests	checking the URL	<code>print('url:', response.url)</code>	<code>g19/demo.py</code>
requests	checking the response text	<code>print(response.text)</code>	<code>g19/demo.py</code>
requests	checking the status code	<code>print('status:', response.status_code)</code>	<code>g19/demo.py</code>
requests	decoding a JSON response	<code>rows = response.json()</code>	<code>g19/demo.py</code>
requests	importing the module	<code>import requests</code>	<code>g19/demo.py</code>
scipy	calling newton's method	<code>cr = opt.newton(find_cube_root,xinit,maxiter=20,args=[y...</code>	<code>g08/demo.py</code>
scipy	importing the module	<code>import scipy.optimize as opt</code>	<code>g08/demo.py</code>
seaborn	adding a title to a grid object	<code>jg.fig.suptitle('Distribution of Hourly Load')</code>	<code>g18/demo.py</code>
seaborn	barplot	<code>hue='month',palette='deep',ax=ax1)</code>	<code>g18/demo.py</code>
seaborn	basic violin plot	<code>sns.violinplot(data=janjul,x="month",y="usage")</code>	<code>g18/demo.py</code>
seaborn	boxenplot	<code>sns.boxenplot(data=janjul,x="month",y="usage")</code>	<code>g18/demo.py</code>
seaborn	calling <code>tight_layout</code> on a grid object	<code>jg.fig.tight_layout()</code>	<code>g18/demo.py</code>
seaborn	drawing a heatmapped grid	<code>sns.heatmap(means,annot=True,fmt="0f",cmap='Spectral',...</code>	<code>g21/demo.py</code>
seaborn	importing the module	<code>import seaborn as sns</code>	<code>g18/demo.py</code>
seaborn	joint distribution hex plot	<code>jg = sns.jointplot(data=bymo,x=1,y=7,kind='hex')</code>	<code>g18/demo.py</code>
seaborn	line plot	<code>sns.lineplot(data=long_form,x='hour',y='value',hue='mon...</code>	<code>g18/demo.py</code>
seaborn	setting axis titles on a grid object	<code>jg.set_axis_labels('January','July')</code>	<code>g18/demo.py</code>
seaborn	setting the theme	<code>sns.set_theme(style="white")</code>	<code>g18/demo.py</code>
seaborn	split violin plot	<code>hue="month",palette='deep',split=True)</code>	<code>g18/demo.py</code>
sys	exiting a script	<code>sys.exit()</code>	<code>g20/demo.py</code>
sys	importing the module	<code>import sys</code>	<code>g20/demo.py</code>
zipfile	importing the module	<code>import zipfile</code>	<code>g16/demo.py</code>
zipfile	opening a file in an archive	<code>fh1 = archive.open('generators-oswego.csv')</code>	<code>g16/demo.py</code>
zipfile	opening an archive	<code>archive = zipfile.ZipFile('generators.zip')</code>	<code>g16/demo.py</code>
zipfile	reading the list of files	<code>print(archive.namelist())</code>	<code>g16/demo.py</code>